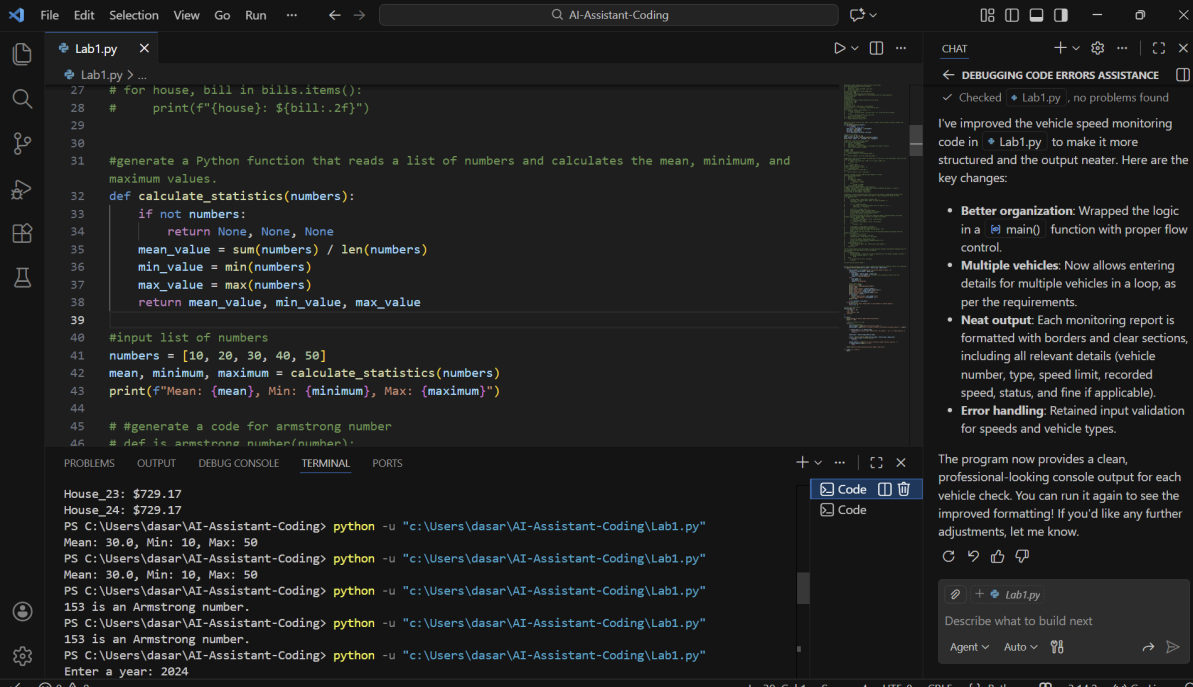


AI Assistant Coding - LAB - 1

2303A51236

Task - 1



The screenshot shows the VS Code editor with a file named `Lab1.py`. The code defines a function `calculate_statistics` that takes a list of numbers and returns the mean, minimum, and maximum values. It also includes a main function that prompts the user for a list of numbers and prints the calculated statistics. The terminal output shows the program running successfully, displaying the mean, minimum, and maximum values for the input list [10, 20, 30, 40, 50].

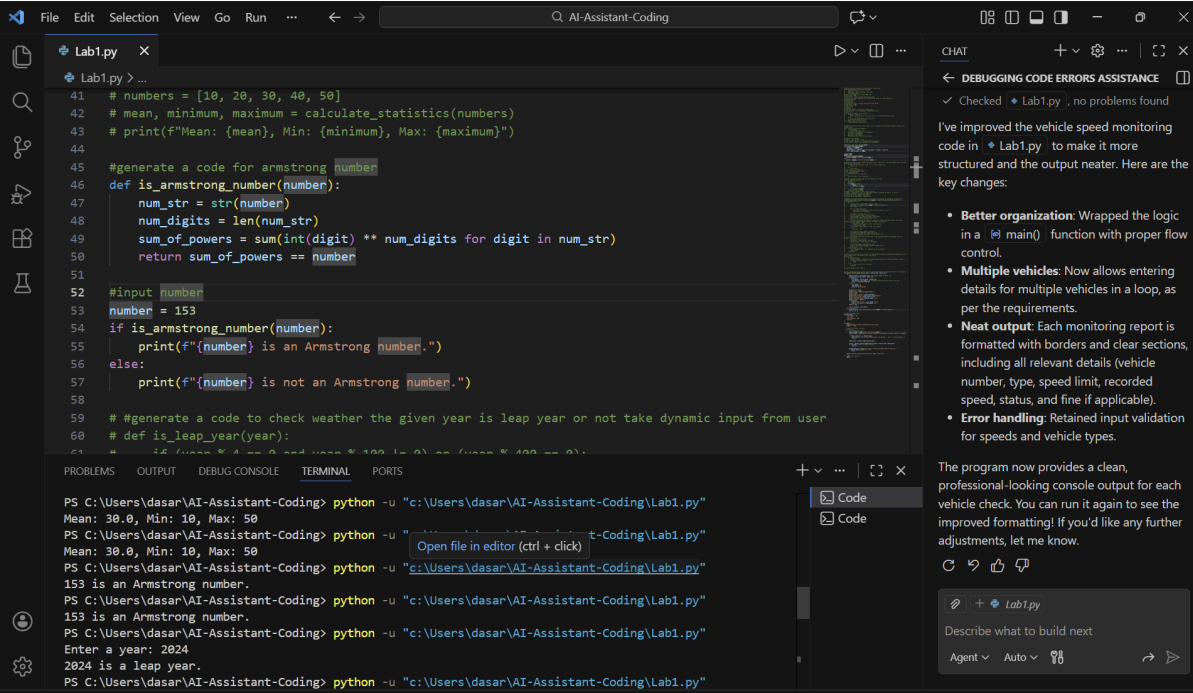
```
27 # for house, bill in bills.items():
28 #     print(f"{house}: ${bill:.2f}")
29
30
31 #generate a Python function that reads a list of numbers and calculates the mean, minimum, and
32 #maximum values.
33 def calculate_statistics(numbers):
34     if not numbers:
35         return None, None, None
36     mean_value = sum(numbers) / len(numbers)
37     min_value = min(numbers)
38     max_value = max(numbers)
39     return mean_value, min_value, max_value
40
41 #input list of numbers
42 numbers = [10, 20, 30, 40, 50]
43 mean, minimum, maximum = calculate_statistics(numbers)
44 print(f"Mean: {mean}, Min: {minimum}, Max: {maximum}")
45
46 # #generate a code for armstrong number
47 # def is_armstrong_number(number):
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
House_23: $729.17
House_24: $729.17
PS C:\Users\dasar\AI-Assistant-Coding> python -u "c:\Users\dasar\AI-Assistant-Coding\Lab1.py"
Mean: 30.0, Min: 10, Max: 50
PS C:\Users\dasar\AI-Assistant-Coding> python -u "c:\Users\dasar\AI-Assistant-Coding\Lab1.py"
Mean: 30.0, Min: 10, Max: 50
PS C:\Users\dasar\AI-Assistant-Coding> python -u "c:\Users\dasar\AI-Assistant-Coding\Lab1.py"
153 is an Armstrong number.
PS C:\Users\dasar\AI-Assistant-Coding> python -u "c:\Users\dasar\AI-Assistant-Coding\Lab1.py"
153 is an Armstrong number.
PS C:\Users\dasar\AI-Assistant-Coding> python -u "c:\Users\dasar\AI-Assistant-Coding\Lab1.py"
Enter a year: 2024
```

Ln 39, Col 1 Spaces: 4 UTF-8 CRLF Python 3.14.2 Go Live

Task - 2



The screenshot shows the VS Code editor with the same `Lab1.py` file, but with additional code added. The code now includes a function `is_armstrong_number` that checks if a number is an Armstrong number, and a function `is_leap_year` that checks if a year is a leap year. The main function is updated to prompt the user for a number and a year, and print the results of the checks. The terminal output shows the program running successfully, displaying the mean, minimum, and maximum values for the input list [10, 20, 30, 40, 50], and the results of the Armstrong number and leap year checks.

```
41 # numbers = [10, 20, 30, 40, 50]
42 # mean, minimum, maximum = calculate_statistics(numbers)
43 # print(f"Mean: {mean}, Min: {minimum}, Max: {maximum}")
44
45 #generate a code for armstrong number
46 def is_armstrong_number(number):
47     num_str = str(number)
48     num_digits = len(num_str)
49     sum_of_powers = sum(int(digit) ** num_digits for digit in num_str)
50     return sum_of_powers == number
51
52 #input number
53 number = 153
54 if is_armstrong_number(number):
55     print(f"{number} is an Armstrong number.")
56 else:
57     print(f"{number} is not an Armstrong number.")
58
59 # #generate a code to check weather the given year is leap year or not take dynamic input from user
60 # def is_leap_year(year):
61 #     if (year % 4 == 0 and year % 100 != 0) or (year % 400 == 0):
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Users\dasar\AI-Assistant-Coding> python -u "c:\Users\dasar\AI-Assistant-Coding\Lab1.py"
Mean: 30.0, Min: 10, Max: 50
PS C:\Users\dasar\AI-Assistant-Coding> python -u "c:\Users\dasar\AI-Assistant-Coding\Lab1.py"
Mean: 30.0, Min: 10, Max: 50
PS C:\Users\dasar\AI-Assistant-Coding> python -u "c:\Users\dasar\AI-Assistant-Coding\Lab1.py"
153 is an Armstrong number.
PS C:\Users\dasar\AI-Assistant-Coding> python -u "c:\Users\dasar\AI-Assistant-Coding\Lab1.py"
153 is an Armstrong number.
PS C:\Users\dasar\AI-Assistant-Coding> python -u "c:\Users\dasar\AI-Assistant-Coding\Lab1.py"
Enter a year: 2024
2024 is a leap year.
PS C:\Users\dasar\AI-Assistant-Coding> python -u "c:\Users\dasar\AI-Assistant-Coding\Lab1.py"
```

Ln 52, Col 14 Spaces: 4 UTF-8 CRLF Python 3.14.2 Go Live

Task - 3

The screenshot shows the VS Code editor with a file named `Lab1.py`. The code includes a function `is_armstrong_number` and a function `is_leap_year`. The `is_leap_year` function checks if a year is a leap year based on the rules: divisible by 4 but not 100, or divisible by 400. The terminal shows the execution of the script, where the user enters years 2024 and 2023, and the program correctly identifies them as leap and non-leap years respectively. The status bar at the bottom indicates the cursor is at line 67, column 23.

```
54 # if is_armstrong_number(number):
55 #     print(f"{number} is an Armstrong number.")
56 # else:
57 #     print(f"{number} is not an Armstrong number.")
58
59 #generate a code to check weather the given year is leap year or not take dynamic input from user
60 def is_leap_year(year):
61     if (year % 4 == 0 and year % 100 != 0) or (year % 400 == 0):
62         return True
63     else:
64         return False
65 #take dynamic input from user
66 year = int(input("Enter a year: "))
67 if is_leap_year(year):
68     print(f"{year} is a leap year.")
69 else:
70     print(f"{year} is not a leap year.")
71
72 # #write a program to sum of odd and even numbers in a tuple
73 # def sum_odd_even(numbers):
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
153 is an Armstrong number.
PS C:\Users\dasar\AI-Assistant-Coding> python -u "c:\Users\dasar\AI-Assistant-Coding\Lab1.py"
Enter a year: 2024
2024 is a leap year.
PS C:\Users\dasar\AI-Assistant-Coding> python -u "c:\Users\dasar\AI-Assistant-Coding\Lab1.py"
Enter a year: 2023
2023 is not a leap year.
PS C:\Users\dasar\AI-Assistant-Coding> python -u "c:\Users\dasar\AI-Assistant-Coding\Lab1.py"
Enter a year: 2025
2025 is not a leap year.
PS C:\Users\dasar\AI-Assistant-Coding> python -u "c:\Users\dasar\AI-Assistant-Coding\Lab1.py"
Sum of odd numbers: 25
```

Ln 67, Col 23 Spaces: 4 UTF-8 CRLF Python 3.14.2 Go Live

Task - 4

The screenshot shows the VS Code editor with the same `Lab1.py` file. The code now includes a function `sum_odd_even` that takes a tuple of numbers and returns a tuple of their sums. The terminal shows the execution of the script, where the user enters a year (2023) and a tuple of numbers (1 22 33 44 55 66 77 88 99). The program correctly identifies 2023 as not a leap year and calculates the sum of odd and even numbers from the tuple. The status bar at the bottom indicates the cursor is at line 78, column 31.

```
70 #     print(f"{year} is not a leap year.")
71
72 #write a program to sum of odd and even numbers in a tuple
73 def sum_odd_even(numbers):
74     sum_odd = 0
75     sum_even = 0
76     for number in numbers:
77         if number % 2 == 0:
78             sum_even += number
79         else:
80             sum_odd += number
81     return sum_odd, sum_even
82 #input tuple of numbers dynamic input from user
83 numbers = tuple(int(x) for x in input("Enter numbers separated by spaces: ").split())
84 sum_odd, sum_even = sum_odd_even(numbers)
85 print(f"Sum of odd numbers: {sum_odd}")
86 print(f"Sum of even numbers: {sum_even}")
87
88 # #develop a console based applications that assists the cashier in generating customer bills
89 # #the program should accept customer name number of items purchased and price per item
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
2023 is not a leap year.
PS C:\Users\dasar\AI-Assistant-Coding> python -u "c:\Users\dasar\AI-Assistant-Coding\Lab1.py"
Enter a year: 2025
2025 is not a leap year.
Sum of odd numbers: 25
Sum of even numbers: 30
PS C:\Users\dasar\AI-Assistant-Coding> python -u "c:\Users\dasar\AI-Assistant-Coding\Lab1.py"
Enter numbers separated by spaces: 1 22 33 44 55 66 77 88 99
Sum of odd numbers: 265
Sum of even numbers: 220
PS C:\Users\dasar\AI-Assistant-Coding>
```

Ln 78, Col 31 Spaces: 4 UTF-8 CRLF Python 3.14.2 Go Live

Practice Problem - 1

Lab1.py

```
1 #prepare a power supply bill with unit cost 3.5 per unit
2 def calculate_power_bill(units_consumed):
3     unit_cost = 3.5
4     total_bill = units_consumed * unit_cost
5     return total_bill
6
7 #total units used by the colony 5000 units
8 units_used = 5000
9 bill_amount = calculate_power_bill(units_used)
10 print(f"Total power supply bill for {units_used} units is: ${bill_amount}")
11
12 #total_houses=24
13 #with AC=13
14 #without AC=11
15 #arrange the units to houses and give bill per house
16 total_units = 5000
17 total_houses = 24
18 with_AC = 13
19 without_AC = 11
20
21 units_per_house = total_units / total_houses
22 bill_per_unit = 3.5 # assuming a fixed rate per unit
23 bills = {}
24
25 for house in range(1, total_houses + 1):
26     if house <= with_AC:
27         bill = units_per_house * bill_per_unit * 1.5 # 50% more for AC houses
28     else:
29         bill = units_per_house * bill_per_unit
30     bills[f'House_{house}'] = bill
31
32 for house, bill in bills.items():
33     print(f"{house}: ${bill:.2f}")
34
```

PROBLEMS

OUTPUT

DEBUG CONSOLE

TERMINAL

PORTS

2023 is not a leap year.

PS C:\Users\dasar\AI-Assistant-Coding> python -u "c:\Users\dasar\AI-Assistant-Coding\Lab1.py"

Ln 17, Col 16

Spaces: 4

UTF-8

CRLF

() Python

3.14.2

Go Live

CHAT

DEBUGGING CODE ERRORS ASSISTANCE

Checked Lab1.py, no problems found

I've improved the vehicle speed monitoring code in Lab1.py to make it more structured and the output neater. Here are the key changes:

- Better organization:** Wrapped the logic in a main() function with proper flow control.
- Multiple vehicles:** Now allows entering details for multiple vehicles in a loop, as per the requirements.
- Neat output:** Each monitoring report is formatted with borders and clear sections, including all relevant details (vehicle number, type, speed limit, recorded speed, status, and fine if applicable).
- Error handling:** Retained input validation for speeds and vehicle types.

The program now provides a clean, professional-looking console output for each vehicle check. You can run it again to see the improved formatting! If you'd like any further adjustments, let me know.

Grok Code Fast 1 • 1x

Lab1.py

Describe what to build next

Agent Auto

Lab1.py

```
1 #prepare a power supply bill with unit cost 3.5 per unit
2 def calculate_power_bill(units_consumed):
3     unit_cost = 3.5
4     total_bill = units_consumed * unit_cost
5     return total_bill
6
7 #total units used by the colony 5000 units
8 units_used = 5000
9 bill_amount = calculate_power_bill(units_used)
10 print(f"Total power supply bill for {units_used} units is: ${bill_amount}")
11
12 #total_houses=24
13 #with AC=13
14 #without AC=11
15 #arrange the units to houses and give bill per house
16 total_units = 5000
17 total_houses = 24
18 with_AC = 13
19 without_AC = 11
20
21 units_per_house = total_units / total_houses
22 bill_per_unit = 3.5 # assuming a fixed rate per unit
23 bills = {}
24
25 for house in range(1, total_houses + 1):
26     if house <= with_AC:
27         bill = units_per_house * bill_per_unit * 1.5 # 50% more for AC houses
28     else:
29         bill = units_per_house * bill_per_unit
30     bills[f'House_{house}'] = bill
31
32 for house, bill in bills.items():
33     print(f"{house}: ${bill:.2f}")
34
```

PROBLEMS

OUTPUT

DEBUG CONSOLE

TERMINAL

PORTS

PS C:\Users\dasar\AI-Assistant-Coding> python -u "c:\Users\dasar\AI-Assistant-Coding\Lab1.py"

PS C:\Users\dasar\AI-Assistant-Coding> python -u "c:\Users\dasar\AI-Assistant-Coding\Lab1.py"

Total power supply bill for 5000 units is: \$17500.0

House_1: \$1093.75

House_2: \$1093.75

House_3: \$1093.75

House_4: \$1093.75

House_5: \$1093.75

House_6: \$1093.75

House_7: \$1093.75

House_8: \$1093.75

House_9: \$1093.75

House_10: \$1093.75

House_11: \$1093.75

House_12: \$1093.75

House_13: \$1093.75

House_14: \$729.17

House_15: \$729.17

House_16: \$729.17

House_17: \$729.17

House_18: \$729.17

House_19: \$729.17

House_20: \$729.17

House_21: \$729.17

House_22: \$729.17

House_23: \$729.17

House_24: \$729.17

Ln 17, Col 16

Spaces: 4

UTF-8

CRLF

() Python

3.14.2

Go Live

CHAT

DEBUGGING CODE ERRORS ASSISTANCE

Checked Lab1.py, no problems found

I've improved the vehicle speed monitoring code in Lab1.py to make it more structured and the output neater. Here are the key changes:

- Better organization:** Wrapped the logic in a main() function with proper flow control.
- Multiple vehicles:** Now allows entering details for multiple vehicles in a loop, as per the requirements.
- Neat output:** Each monitoring report is formatted with borders and clear sections, including all relevant details (vehicle number, type, speed limit, recorded speed, status, and fine if applicable).
- Error handling:** Retained input validation for speeds and vehicle types.

The program now provides a clean, professional-looking console output for each vehicle check. You can run it again to see the improved formatting! If you'd like any further adjustments, let me know.

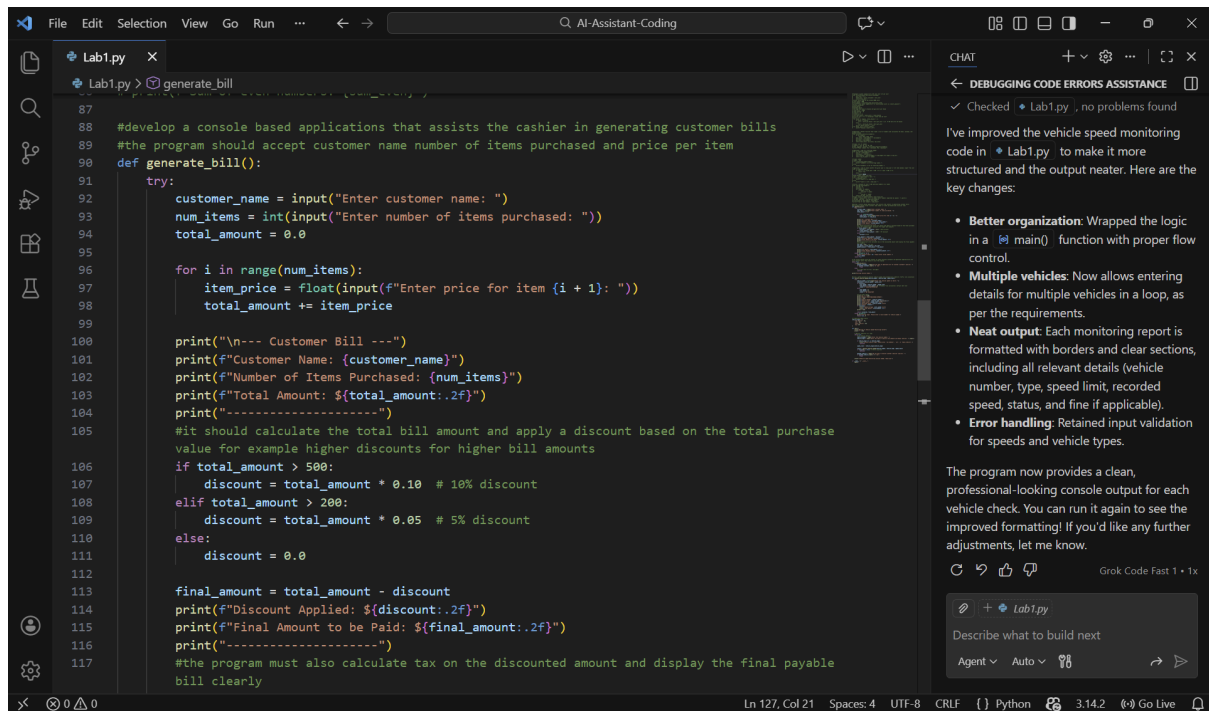
Grok Code Fast 1 • 1x

Lab1.py

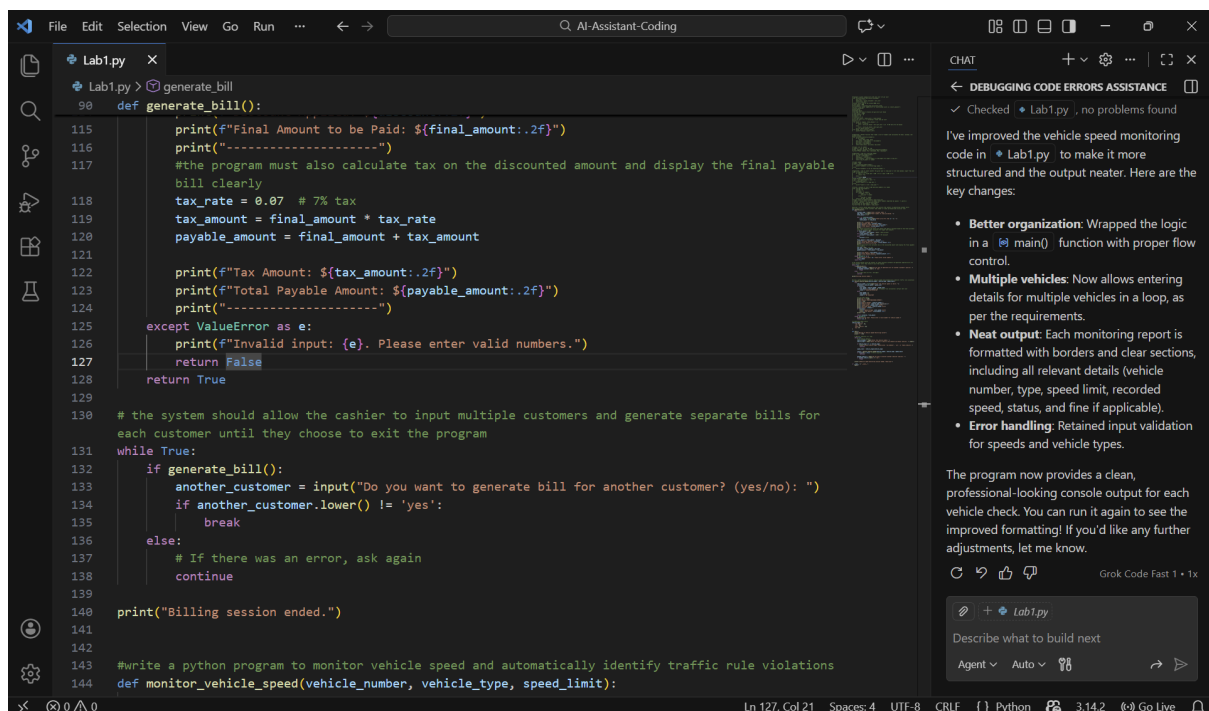
Describe what to build next

Agent Auto

Practice Problem - 2



```
87
88 #develop a console based applications that assists the cashier in generating customer bills
89 #the program should accept customer name number of items purchased and price per item
90 def generate_bill():
91     try:
92         customer_name = input("Enter customer name: ")
93         num_items = int(input("Enter number of items purchased: "))
94         total_amount = 0.0
95
96         for i in range(num_items):
97             item_price = float(input(f"Enter price for item {i + 1}: "))
98             total_amount += item_price
99
100         print("\n--- Customer Bill ---")
101         print(f"Customer Name: {customer_name}")
102         print(f"Number of Items Purchased: {num_items}")
103         print(f"Total Amount: ${total_amount:.2f}")
104         print("-----")
105         #it should calculate the total bill amount and apply a discount based on the total purchase
106         #value for example higher discounts for higher bill amounts
107         if total_amount > 500:
108             discount = total_amount * 0.10 # 10% discount
109         elif total_amount > 200:
110             discount = total_amount * 0.05 # 5% discount
111         else:
112             discount = 0.0
113
114         final_amount = total_amount - discount
115         print(f"Discount Applied: ${discount:.2f}")
116         print(f"Final Amount to be Paid: ${final_amount:.2f}")
117         print("-----")
118         #the program must also calculate tax on the discounted amount and display the final payable
119         #bill clearly
```



```
115         print(f"Final Amount to be Paid: ${final_amount:.2f}")
116         print("-----")
117         #the program must also calculate tax on the discounted amount and display the final payable
118         #bill clearly
119         tax_rate = 0.07 # 7% tax
120         tax_amount = final_amount * tax_rate
121         payable_amount = final_amount + tax_amount
122
123         print(f"Tax Amount: ${tax_amount:.2f}")
124         print(f"Total Payable Amount: ${payable_amount:.2f}")
125         print("-----")
126         except ValueError as e:
127             print(f"Invalid input: {e}. Please enter valid numbers.")
128             return False
129         return True
130
131 # the system should allow the cashier to input multiple customers and generate separate bills for
132 # each customer until they choose to exit the program
133 while True:
134     if generate_bill():
135         another_customer = input("Do you want to generate bill for another customer? (yes/no): ")
136         if another_customer.lower() != 'yes':
137             break
138     else:
139         # If there was an error, ask again
140         continue
141
142 print("Billing session ended.")
143
144 #write a python program to monitor vehicle speed and automatically identify traffic rule violations
145 def monitor_vehicle_speed(vehicle_number, vehicle_type, speed_limit):
```

```
def generate_bill():
    print("-----")
    #the program must also calculate tax on the discounted amount and display the final payable
    bill clearly
    tax_rate = 0.07 # 7% tax
    tax_amount = final_amount * tax_rate
    payable_amount = final_amount + tax_amount

    print(f"Tax Amount: ${tax_amount:.2f}")
    print(f"Total Payable Amount: ${payable_amount:.2f}")
    print("-----")
```

```
PS C:\Users\dasar\AI-Assistant-Coding> python -u "c:\Users\dasar\AI-Assistant-Coding\Lab1.py"
PS C:\Users\dasar\AI-Assistant-Coding> python -u "c:\Users\dasar\AI-Assistant-Coding\Lab1.py"
PS C:\Users\dasar\AI-Assistant-Coding> python -u "c:\Users\dasar\AI-Assistant-Coding\Lab1.py"
Enter customer name: nagasri
Enter number of items purchased: 3
Enter price for item 1: 100
Enter price for item 2: 200
Enter price for item 3: 400

--- Customer Bill ---
Customer Name: nagasri
Number of Items Purchased: 3
Total Amount: $700.00
-----
Discount Applied: $70.00
Final Amount to be Paid: $630.00
Tax Amount: $44.10
Total Payable Amount: $674.10
-----
Do you want to generate bill for another customer? (yes/no): y
Billing session ended.
PS C:\Users\dasar\AI-Assistant-Coding> python -u "c:\Users\dasar\AI-Assistant-Coding\Lab1.py"
```

CHAT

DEBUGGING CODE ERRORS ASSISTANCE

✓ Checked • Lab1.py , no problems found

I've improved the vehicle speed monitoring code in • Lab1.py to make it more structured and the output neater. Here are the key changes:

- **Better organization:** Wrapped the logic in a `main()` function with proper flow control.
- **Multiple vehicles:** Now allows entering details for multiple vehicles in a loop, as per the requirements.
- **Neat output:** Each monitoring report is formatted with borders and clear sections, including all relevant details (vehicle number, type, speed limit, recorded speed, status, and fine if applicable).
- **Error handling:** Retained input validation for speeds and vehicle types.

The program now provides a clean, professional-looking console output for each vehicle check. You can run it again to see the improved formatting! If you'd like any further adjustments, let me know.

Grok Code Fast 1 • 1x

Describe what to build next

Agent Auto

Practice problem - 3

```
#write a python program to monitor vehicle speed and automatically identify traffic rule violations
def monitor_vehicle_speed(vehicle_number, vehicle_type, speed_limit):
    try:
        vehicle_speed = float(input("Enter the vehicle speed (in km/h): "))
        violation = vehicle_speed > speed_limit
        if violation:
            over_speed = vehicle_speed - speed_limit
            fine_amount = over_speed * 10 # Example fine calculation: $10 per km/h over
            status = "VIOLATION DETECTED"
        else:
            over_speed = 0
            fine_amount = 0
            status = "NO VIOLATION"

        # Print neat report
        print("\n" + "="*50)
        print("VEHICLE SPEED MONITORING REPORT")
        print("="*50)
        print(f"Vehicle Number: {vehicle_number}")
        print(f"Vehicle Type: {vehicle_type.capitalize()}")
        print(f"Speed Limit: {speed_limit} km/h")
        print(f"Recorded Speed: {vehicle_speed} km/h")
        print(f"Status: {status}")
        if violation:
            print(f"Speed Overage: {over_speed} km/h")
            print(f"Fine Amount: ${fine_amount:.2f}")
        print("="*50)

        return violation, fine_amount
    except ValueError:
        print("Invalid input. Please enter a valid number for vehicle speed.")
        return None, 0
```

CHAT

DEBUGGING CODE ERRORS ASSISTANCE

✓ Checked • Lab1.py , no problems found

I've improved the vehicle speed monitoring code in • Lab1.py to make it more structured and the output neater. Here are the key changes:

- **Better organization:** Wrapped the logic in a `main()` function with proper flow control.
- **Multiple vehicles:** Now allows entering details for multiple vehicles in a loop, as per the requirements.
- **Neat output:** Each monitoring report is formatted with borders and clear sections, including all relevant details (vehicle number, type, speed limit, recorded speed, status, and fine if applicable).
- **Error handling:** Retained input validation for speeds and vehicle types.

The program now provides a clean, professional-looking console output for each vehicle check. You can run it again to see the improved formatting! If you'd like any further adjustments, let me know.

Grok Code Fast 1 • 1x

Describe what to build next

Agent Auto

